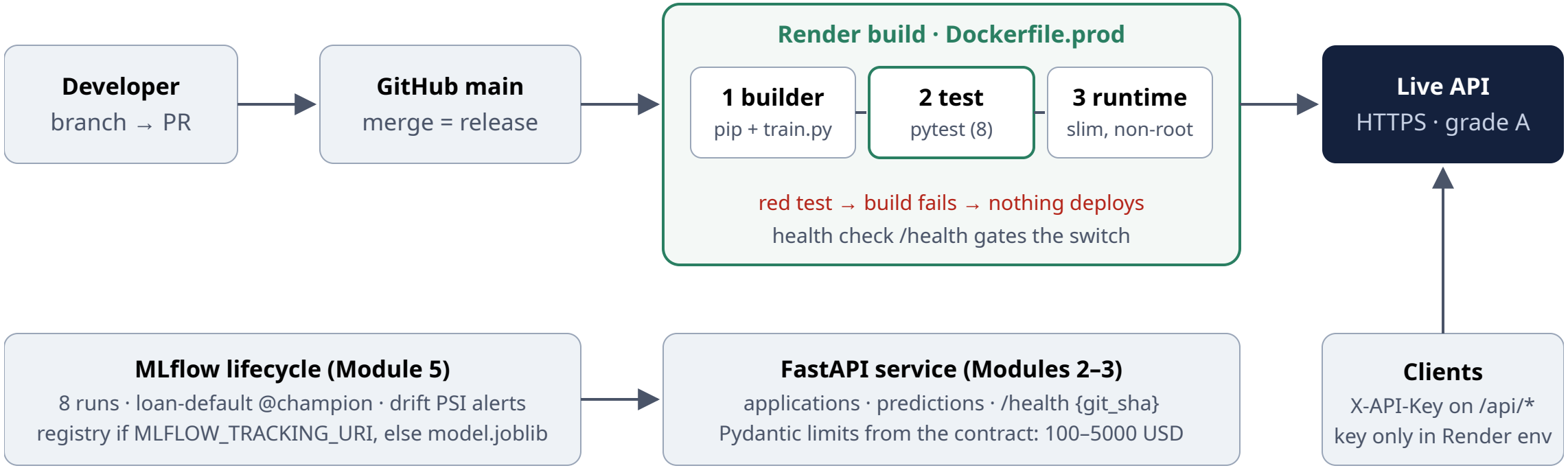
Loan risk API: from code to production

A microfinance default-risk model, served by FastAPI, tested, built and deployed on every merge, and protected by a test for the bug that once shipped.

Sok Cheapanha · loan-api-cheapanha.onrender.com · github.com/Cheapanha-Sok/loan-api-tp2

ARCHITECTURE

Contract → service → image → lifecycle → pipeline



Verify: scripts/verify.sh polls /health until git_sha = merged commit, prints merge-to-live.

Rollback: Render Events → Rollback to an earlier SHA, then fix forward through a PR.

RESULTS

The pipeline in numbers

1:05

merge to live for the real improvement (PR #2, contract's `Location` header)

48 s

rollback to recovered after the bad release (drill part 2)

A

SSL Labs grade, TLS 1.2 and 1.3 only, auto-renewed certificate

0

hand deploys; 0 secrets in Git, image or logs

PR	Change	Outcome
#2	Add `Location` header (contract)	**live in 1 min 5 s**
#3	Drill 1: loan limit 9000	**build failed at pytest, never live**
#5	Drill 2: inverted probability	**live 1 min 27 s, then rolled back**
#6	Fix forward + new test	**live in 1 min 10 s; same bug now fails the build**

Times measured by verify.sh from the merge commit to the live git_sha

ONE DESIGN DECISION I WOULD DEFEND

Run the tests inside the image build

- The `test` stage in `Dockerfile.prod` runs pytest against the **exact code and model file** that will ship.
- The runtime stage copies a marker from it, so a red test means **no image and no deploy**, on any builder.
- Proven in drill 1: PR #3 merged, Render build failed, production stayed on the old SHA.

Why it matters

When GitHub Actions was locked by a billing issue, the release gate did not disappear with it. CI moved into the artifact itself, so the guarantee "only tested code goes live" held without any CI service.

Plus: deploys are verified by commit

`/health` reports `git_sha`; "healthy" alone is not proof, because the old version is healthy too.

ONE TRADE-OFF I ACCEPTED

Render's free Git pipeline instead of the full GitHub pipeline

What I gained

A working, automatic test → build → deploy → health-check pipeline the same day, at zero cost, with HTTPS, logs and one-click rollback, despite the account lock.

What I gave up

No branch protection (it needs a CI status check), no SHA-tagged images in GHCR, and verify runs from my terminal instead of a CI job. Render rebuilds on every deploy, and a rollback switches auto-deploy off until re-enabled.

- Also accepted: free-plan **cold starts** (about a minute after 15 idle minutes) and an **in-memory** application store that resets on every deploy.
- Fine for a course project; a paying lender would get a paid instance and PostgreSQL.

ci.yml and publish.yml stay in the repo, ready for when Actions is unlocked

BREAK AND RECOVER

Tests only catch what they check

Time (UTC)	Event	Safe applicant	Risky applicant
17:08	Before	0.095 · low	1.0 · high
17:10:09	Inverted score live (all 7 tests passed)	**0.905 · low**	**0.0 · high**
17:10:48	Rollback clicked in Render		
17:11:36	Recovered (48 s)	0.095 · low	1.0 · high

The real fix

`test_risky_applicant_scores_higher_than_safe_applicant` : a risky borrower must score higher than a safe one and the band must agree with the probability. It fails on the inverted code (1 failed, 7 passed), so the same mistake can never ship again.

Next steps

- **Restore the GitHub pipeline** once Support lifts the lock: `pipeline.yml`, branch protection requiring `test`, SHA-tagged images in GHCR, and a manual approval environment before production.
- **PostgreSQL** for applications, then the missing contract endpoints: pagination, approve / reject, the borrower view.
- **Hosted MLflow** so the live API loads `@champion` from the registry and promotions need no rebuild.
- **Automate the drift alerts**: a daily job computing PSI from live traffic, alerting at 0.1 / 0.2.
- **More model-behaviour tests** (monotonic risk in late payments and income), the class of bug the drill exposed.